Week 01

- ❑ Welcome
- ❑ The Goal of 341
- ❑ 341 Structure
- ❑ The Recipe
- ❑ Intro to OCaml

Lecture 01

Welcome, OCaml Intro

Welcome!

10 weeks to learn *some fundamental concepts* of Programming Languages

- Become better programmers, even in languages we won't use
- Learn to distinguish surface differences from deeper principles
- Develop a systematic methodology for what programs mean
- Pick up some technical jargon to support further study

What is going to happen?

- Bad course summary:
  - “Learn OCaml + Implement Toy Language”
- Better course summary:
  - Precisely characterize programming languages and programs
  - Understand Functional Programming (FP)
  - Use functional programming effectively
  - Understand static types vs. dynamic types
  - Reason precisely about *scope* in many programming settings
  - ...

Today

1. Course mechanics
2. A little motivation and mindset
3. Our first OCaml program

TODO

- Read the course website, especially the syllabus
- Sign in to the Discussion Board
  - You **must** receive announcements (also emailed)
- Set up your programming environment for OCaml
- Complete “Homework 0” (very short, due Friday, 5pm)

About Me: Anjali

- 3rd year PhD student, advised by Zach Tatlock
  - Compilers, verification, equality saturation
  - Motivated by teaching, especially classes like PL
- 5 years as a software engineer before PhD
  - Google + Code.org

TAs

- S Harris
  - OH : Fridays 11:00 - 12:00
- Vova Trifonov
  - OH : Mondays 4:30 - 5:30

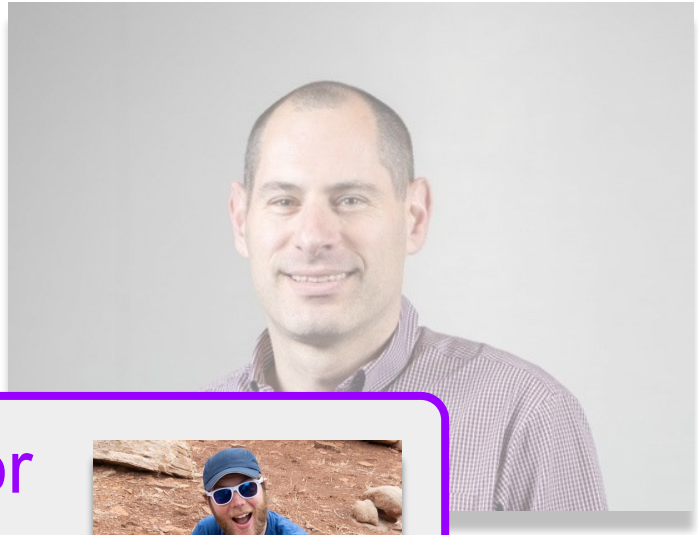
Course Creator

Dan Grossman

- Eminent PL researcher
- Vice Director of the Allen School
- Highly successful Coursera PL course
- Cornering the market on Internet Dad Jokes (@djg98115)

Course Creator

Dan Grossman



- Eminent PL researcher
- Vice D
- Highly
- Corne

Another Course Innovator

James Wilcox

- Another eminent PL researcher!
- 341 OCaml port, Trefoil inventor
- Lots of great new ideas
- Buckle up 😂

Staying in Touch

- Asynchronous communication through Ed
  - Can use “private threads” to communicate directly with staff
- Lecture, Section, Office Hours
  - Show up, ask questions, hang out!
- Please be patient, kind, and helpful 🤝

Lecture

- Recordings: available to the class, but not public
  - Intended for review, backup, and emergencies
  - Occasionally there may not be recordings (e.g., technical issues)
- Take notes! Preferably with pen and paper first 📝
- Interaction is more fun for everybody!
  - Takes the whole class to make good experience for everyone
- Attend! You *will* learn more *and* enjoy class more *and* get a better grade
  - You can't binge-watch this course – needs absorption

Section

- Smaller group meetings, presents new material
  - Often needed to complete homework assignments
- Not recorded, but slides and code will be posted
- Section work is graded by completion
- Meets on Thursdays
- Get to know your TAs!

Office Hours

- Schedule will be on course website
- Feel free to drop in just to chat!
  - Would love to meet everyone this quarter 👋
  - Ideally for chatting about interesting ideas from the material
  - Ideally not *just* for homework questions (but those are OK too)

Homework

- 8 total, mostly programming
- **DO** discuss with friends and classmates *at a high-level*
- **DO NOT** look at or share any code!
    - No AI code generation (ChatGPT, copilot, etc)
- In other words, “individual assignments”

Homework

- Start early! Some problems need sleep
- Deadlines are **strict**: **5pm PT on Fridays**
  - Self Serve 48-hour extension
  - Email me for custom extensions for extenuating circumstances
- Challenge problems: for the fun and interest of it
  - Poor effort-to-points tradeoff
  - Good effort-to-learning tradeoff
  - Don’t attempt challenges till everything else is done!

Homework Strategy

Doing the homework involves:

1. Understanding the concepts being addressed
2. Writing code demonstrating understanding of the concepts
3. Testing your code to ensure you understand and have correct programs
4. Exploring variations, fixing incorrect answers, etc.

We mostly grade 2, but ignoring 1, 3, 4 makes the homework **much** harder!

Yet more advice, from a lot of 341 experience

- Course material is a “grand vision”
  - Concepts compose, making code elegant, powerful, well-defined
  - Material builds on everything that precedes it
- Needs an *entirely different mindset* from “thrash and search for enough random facts to get your homework done.”
- The thrash-and-search approach:
  - Makes the homework far less enjoyable and educational
  - Leads to poor exams
  - Misses the whole point of the course

Academic Integrity

- The rules are to help you learn and to be fair to everyone
- Most cheating arises from procrastination followed by panic
  - Much better to reach out to course staff for help instead!
  - If you collaborate too closely, state that in your submission!
- Don’t believe everything online
  - The homework is not web search, peeking is cheating
  - **DO NOT** post your solutions anywhere
- No code from Copilot, ChatGPT, etc.

Exams

- We won’t deprive you of the opportunity to review and learn
- Especially in ways that coding assignments don’t capture
  - Seriously!

- Plan for 3 quizzes, during section
  - July 17, August 7, August 21

Questions about course mechanics?

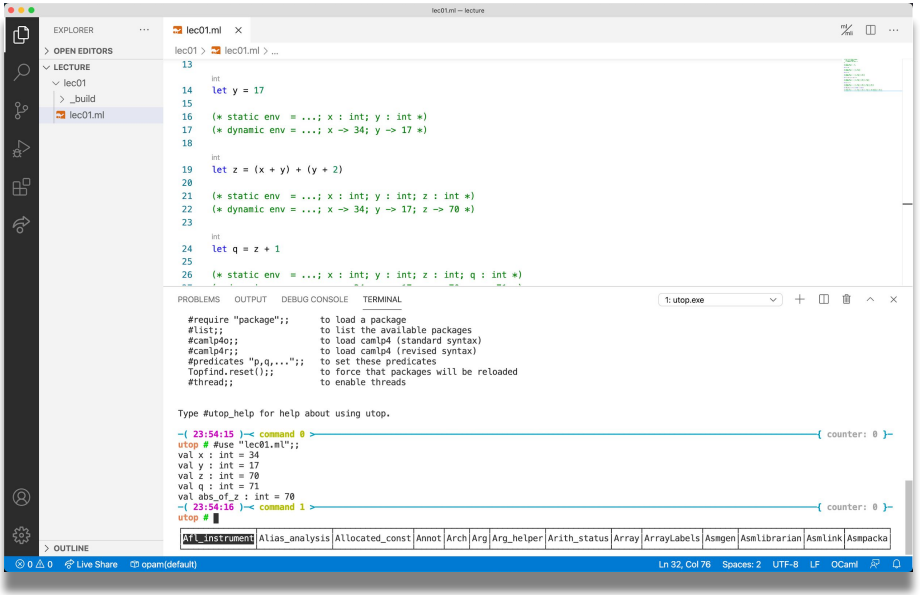
Group Work

- Introductions
  - Name
  - Year
  - Why you're taking this course
- What does "functional programming" refer to? It's okay if you're not sure!!

Programming Languages

- Fundamentals: design, implementation, theory, practice
- View fundamentals through OCaml and Trefoil
  - Learn a lot from seeing the same idea in different guise
  - “Travel to see where you’re from”
- Focus on functional programming (FP)
  - *Avoid mutation* (assignment statements)
  - *Functions are values*

A Whole New World



- New language (OCaml)
- New editor (VS Code, etc.)
- New mode of exploring (REPL)
- Let's get set up and start learning the workflow
    - Sooner the better -- do *not* delay fighting installation demons

Mindset

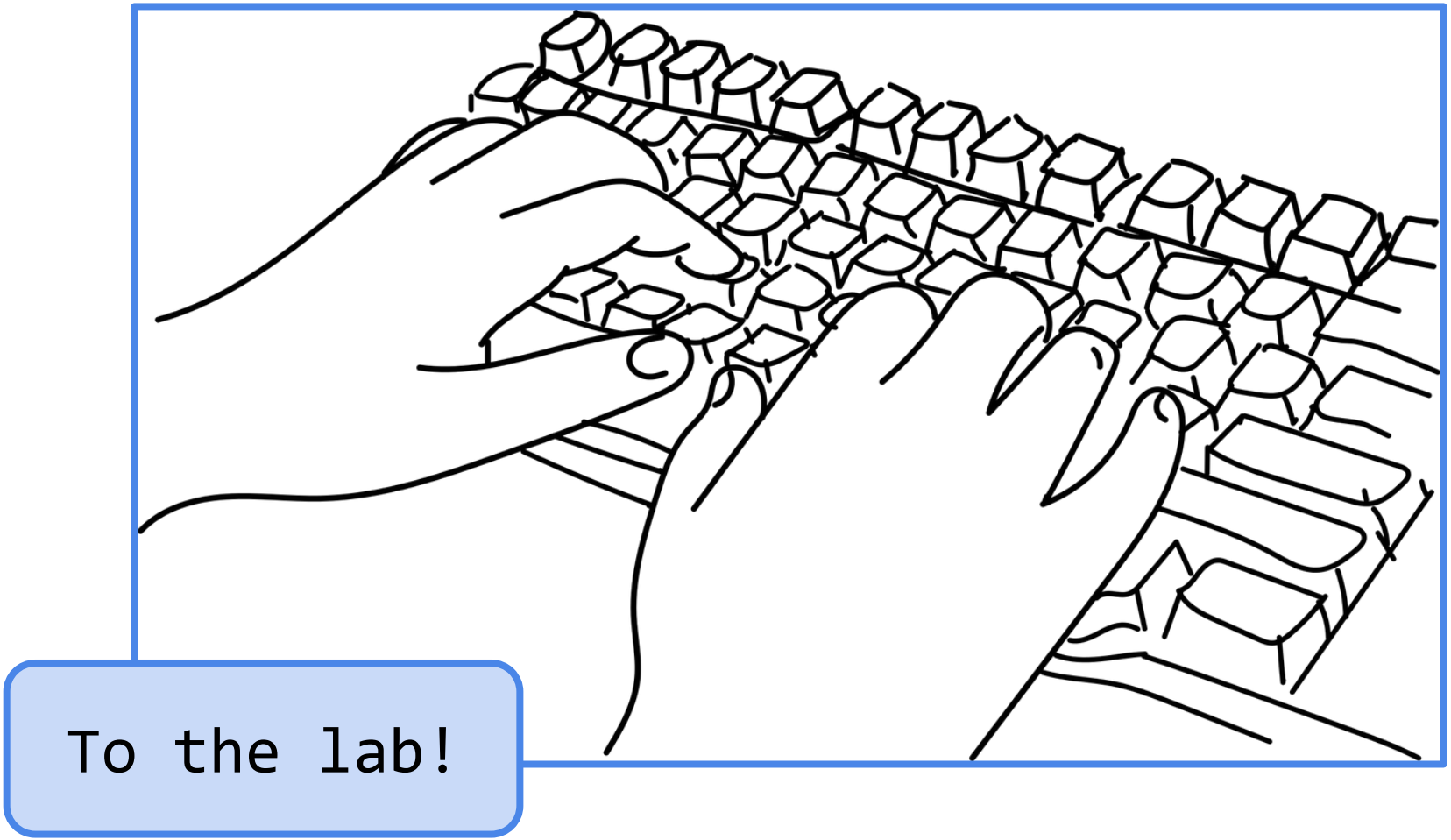
- It’s too easy to forget how mind-bending programming first was
  - *“When you're thinking about something that you don't understand, you have a terrible, uncomfortable feeling called confusion.”*
  - Learning new things often means working through confusion: that’s OK!!
- It’s too easy to think everything is a nail and we only need hammers
  - *“We become what we behold. We shape our tools and then our tools shape us.”*
  - Learning diverse ideas makes us robust when facing new problems!

Mindset

- “Let go” of everything you know
  - Ideas will apply to the mainstream (Java, C#, JS) eventually!
- Treat OCaml (“ML”) as a new alien game we’re learning to play 👽 🎲
  - We will describe OCaml’s core *systematically* (The Recipe)
  - Then we will apply the same system repeatedly
  - The Recipe supports learning many new concepts
  - The Recipe enables comparing / contrasting PLs more rigorously
  - **Trying to “translate everything into Java” will hold you back**

The Recipe: Syntax + Semantics

- Syntax: how programs are written down
  - Roughly “spelling and grammar”
- Semantics: what programs ***mean***
  - Type checking : compile time (aka “static”) semantics
  - Evaluation : run time (aka “dynamic”) semantics
- Both syntax and semantics matter
  - 341’s view ≈ “Semantics give the essence of a PL.”
  - 341’s view ≈ “Syntax is more about taste.”


To the lab!

Defining Variable Binding

`let x = e`

Syntax:

- Keyword: `let`
- Variable: `x`
- Expression: `e`
  - Many forms!
  - Defined *recursively*!

```
(* static env = ... *)
(* dynamic env = ... *)
let x = 34
(* static env  = ...; x : int *)
(* dynamic env = ...; x = 34 *)
```

Defining Variable Binding

```
(* static env = ... *)
(* dynamic env = ... *)
let x = 34
(* static env  = ...; x : int *)
(* dynamic env = ...; x = 34 *)
```

`let x = e`

Semantics:

- Type checking
  - Type check `e` with some type `t`
  - Add binding `x` : `t` to the static environment
- Evaluation
  - Evaluate `e` to some value `v`
  - Add binding `x` ↦ `v` to the dynamic environment

Defining Variable Binding

```
(* static env = ... *)
(* dynamic env = ... *)
let x = 34
(* static env  = ...; x : int *)
(* dynamic env = ...; x = 34 *)
```

`let x = e`

Semantics:

- Type checking
  - Type check `e` with some type `t`
  - Add binding `x` : `t` to the static environment
- Evaluation
  - Evaluate `e` to some value `v`
  - Add binding `x` ↦ `v` to the dynamic environment

metavariables in blue

OCaml Carefully So Far

- *A program is just a sequence of bindings*
- Typecheck each binding in order
  - Using **static environment** from previous bindings
- Evaluate each binding in order
  - Using **dynamic environment** from previous bindings
- So far, only variable bindings
  - More kinds of bindings coming soon!